

MicroC (Part 2)... Creating and using MC09

 flexonsbd.blogspot.com/2018/07/microcmc09.html

Tuesday, July 24, 2018

Using MicroC (Part 2)... Creating and using MC09

More detailed instructions on how to create MC09

In "Using MicroC (Published May 4th)," I explained how to create an MC09, but that was based on records from when I created it on Ubuntu in 2014.

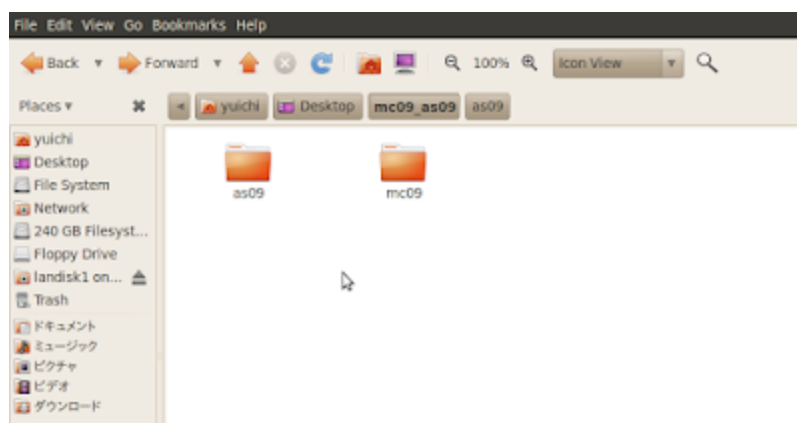
When I wrote it, I thought it might not be possible to create it on the Ubuntu 16 I'm currently using, but I didn't actually confirm if it was impossible.

Afterwards, I tried it on Ubuntu 16 and the latest 18, but I just couldn't get it to work.

So, I installed an older version of Ubuntu and created the MC09 again.

I installed Ubuntu 10, and the ISO file I used was Ubuntu-10.04.4-desktop-i386.iso. (I messed up and broke my environment which was running as a dual boot of Windows 7 and Ubuntu 16... (crying))

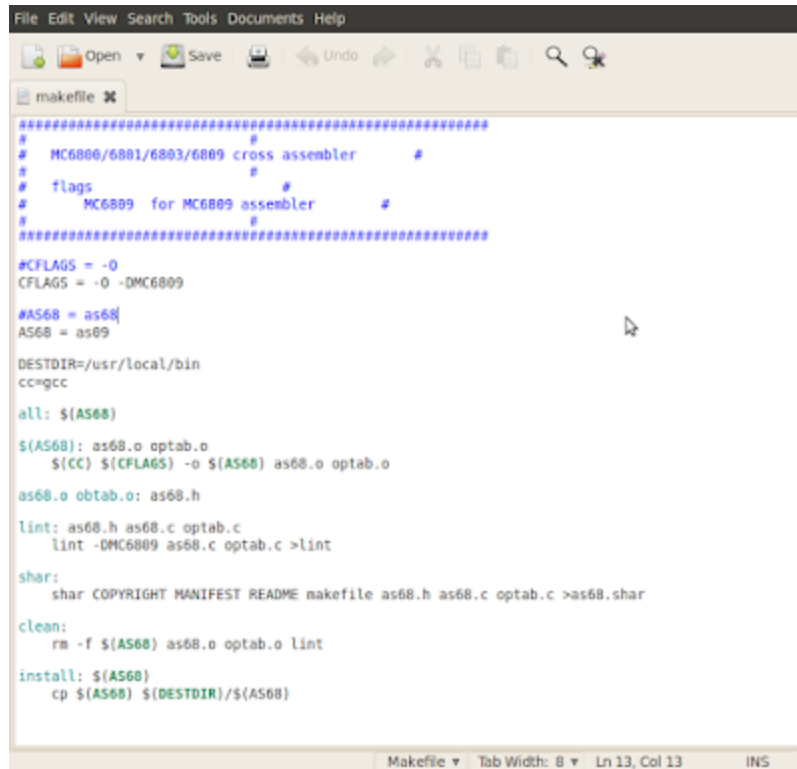
On Ubuntu 10, as explained in the previous post, I downloaded mc09.tgz and as09.tgz from Takeoka's site and extracted them. I created a folder called mc09_as09 on my desktop, and extracted them into folders called mc09 and as09 inside that.



folder containing files

(1) Create the assembler as09 for the 6809.

The makefile is located in the folder as09, and you can basically build it according to its instructions. However, since it's a BSD file, the following modifications are necessary for Linux.

A screenshot of a text editor window titled 'makefile'. The editor shows a makefile with several sections: a header with comments about the cross assembler, flags for the 6809, variable assignments for AS68 and AS09, a DESTDIR variable, and build rules for 'all', 'lint', 'shar', 'clean', and 'install'. The 'all' rule builds 'as68.o' and 'optab.o' using 'cc=gcc' and 'CFLAGS=-O -DMC6809'. The 'lint' rule runs 'lint' on 'as68.h', 'as68.c', and 'optab.c'. The 'shar' rule creates a 'shar' archive. The 'clean' rule removes 'as68.o', 'optab.o', and 'lint'. The 'install' rule copies the files to the destination directory.

```
#####
# MC6806/6801/6803/6809 cross assembler #
# flags #
# MC6809 for MC6809 assembler #
#####
#CFLAGS = -O
CFLAGS = -O -DMC6809

#AS68 = as68
AS68 = as09

DESTDIR=/usr/local/bin
cc=gcc

all: $(AS68)

$(AS68): as68.o optab.o
$(CC) $(CFLAGS) -o $(AS68) as68.o optab.o

as68.o optab.o: as68.h

lint: as68.h as68.c optab.c
lint -DMC6809 as68.c optab.c >lint

shar:
shar COPYRIGHT MANIFEST README makefile as68.h as68.c optab.c >as68.shar

clean:
rm -f $(AS68) as68.o optab.o lint

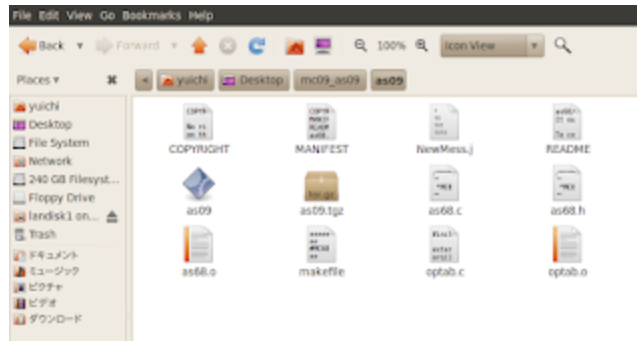
install: $(AS68)
cp $(AS68) $(DESTDIR)/$(AS68)
```

makefile for as09 (after modification)

- Add `cc=gcc` to the line after `CFLAGS=` in the makefile.
- To avoid function name conflicts, rename the following two functions:
 - Change all instances of `getline()` to `getline2()` in as68.c,
 - and change all instances of `index()` to `index2()` in as68.c.

After making the corrections, running `make` will create the executable file as09.

(The lines from `lint:...` onwards in the makefile will not be executed, but this will not affect the result.)



After creating as09

Copy this as09 file to the mc09 folder.

(2) Create mc09

Navigate to the folder mc09. We will work in this folder from now on.

- Add `cc=gcc` on the line after `DESTDIR=`

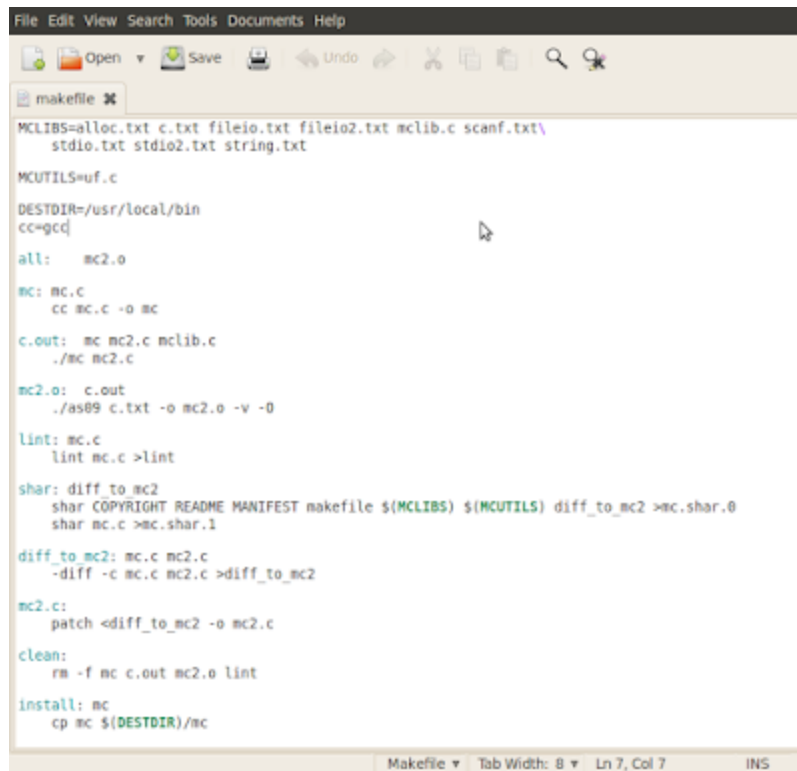
- Add `./` to the beginning of `mc mc2.c: ./mc mc2.c`

Add `./` to the beginning of `as09 c.txt -o mc2.o -v -O: ./as09 c.txt -o mc2.o -v -O`

- To avoid conflicts in function names, change the names of the following two functions:

 - Change all instances of `getline()` in `mc.c` to `getline2()`

 - Change all instances of `index()` in `mc.c` to `index2()`

A screenshot of a text editor window titled 'makefile'. The editor has a menu bar with 'File', 'Edit', 'View', 'Search', 'Tools', 'Documents', and 'Help'. Below the menu is a toolbar with icons for Open, Save, Undo, Redo, Cut, Copy, Paste, Find, and Print. The main text area contains the following makefile content:

```
MCLIBS=alloc.txt c.txt fileio.txt fileio2.txt mclib.c scanf.txt\
stdio.txt stdio2.txt string.txt

MCUTILS=uf.c

DESTDIR=/usr/local/bin
cc=gcc

all: mc2.o

mc: mc.c
    cc mc.c -o mc

c.out: mc mc2.c mclib.c
    ./mc mc2.c

mc2.o: c.out
    ./as09 c.txt -o mc2.o -v -O

lint: mc.c
    lint mc.c >lint

shar: diff_to_mc2
    shar COPYRIGHT README MANIFEST makefile $(MCLIBS) $(MCUTILS) diff_to_mc2 >mc.shar.0
    shar mc.c >mc.shar.1

diff_to_mc2: mc.c mc2.c
    -diff -c mc.c mc2.c >diff_to_mc2

mc2.c:
    patch <diff_to_mc2 -o mc2.c

clean:
    rm -f mc c.out mc2.o lint

install: mc
    cp mc $(DESTDIR)/mc
```

The status bar at the bottom indicates 'Makefile', 'Tab Width: 8', 'Ln 7, Col 7', and 'INS'.

makefile for mc09 (after modification)

With the changes made so far, the `gcc mc.c -o mc` command in the makefile creates the `mc` file.

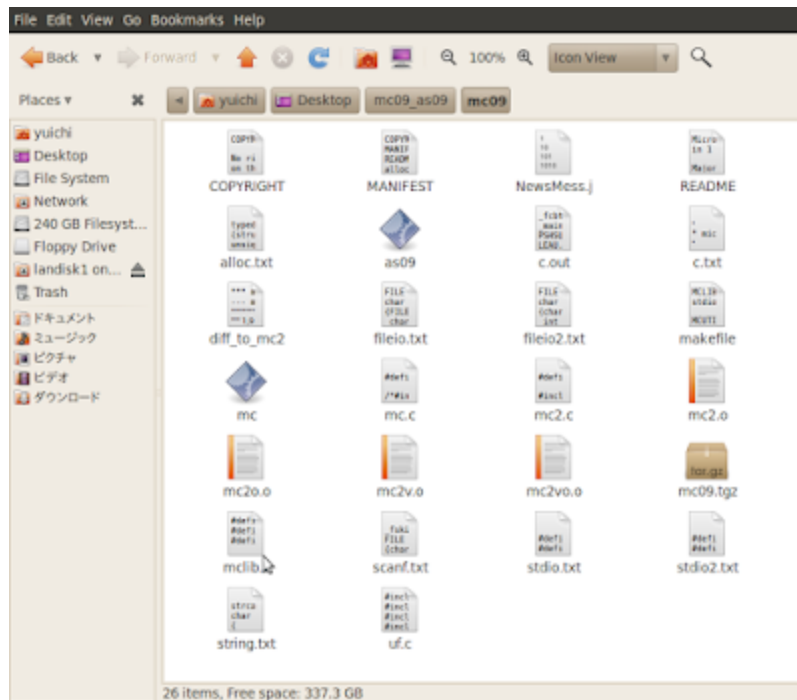
This `mc` can be used as a cross-compiler on Linux, but as it is, while compilation is possible, output (C.OUT) will not be obtained. We will make the following changes.

- Change all instances of `printf(` in `mc.c` to `fprintf(obuf,`).

This change allows

`gcc mc.c -o mc` in the makefile to create the cross-compiler `mc`. Using this `mc`, `./mc mc2.c` compiles `mc2.c`, yielding the result `c.out` (6809 assembler language). Assembling this into 6809 machine code with `./as09 c.txt -o mc2.o -v -O` yields the executable file `mc2.o` (Motorola S format) for Flex. (`c.out` is read and assembled together with `c.txt` during assembly.)

(If `make` is executed instead of line by line, lines after `lint:...` will not be executed, but this does not affect the result.)



After creating four different versions of mc2.o

Finally, convert mc2.o from S format to binary, load it into Flex, and rename it to MC09.CMD, and you're done. I used the previously released CvtMotHexBin.exe to convert it to binary (setting both StartAdr and ExeAdr to 0x0100), and then put it into FlexDrvWin.exe.

Additional Note:

I wasn't sure about the effects of the -v and -O options in the assembler as09, so I created four versions: one with no options (mc2.o), one with only -v (mc2v.o), one with only -O (mc2o.o), and one with both -v and -O (mc2vo.o).

Results: mc2.o and mc2v.o were the same, both 31KB in binary size, and mc2o.o and mc2vo.o were the same, both 29KB. -O seems to optimize size, but I couldn't figure out what -v does.

Finally, as explained last time, copy

C.TXT, and the libraries c.txt, stdio.txt, stdio2.txt, alloc.txt, fileio.txt, fileio2.txt, scanf.txt, string.txt, and the utility uf.c to the Flex's drive, and you're done. (Copy the library to the system disk on drive 0, and C.TXT to the work disk on drive 1.) The file to be compiled is created on the work drive where C.TXT is located. With this arrangement, for example, MC09 (sp) -O1.C.OUT (sp) file.c (sp) is a space . This compiles file.c to create C.OUT (6809 assembly language), and then the assembler

ASMB.CMD on Flex assembles it with
ASMB (sp) C (sp) file.CMD +YLS (C refers to C.TXT),
creating the executable file file.CMD.
This file is placed in \$100 and execution starts from \$100.

Precautions when using MC09

As I wrote last time, I'll write it again, including some additions.

(1) When creating source files

: - Always include stdio.txt and alloc.txt. The file names should be in uppercase.

```
#include "STDIO.TXT"
```

```
#include "ALLOC.TXT" (If you don't include them, you will get an initheap error during  
assembly)
```

- TAB code (\$09) has a different meaning in Flex, so convert it to a space.

- The only line ending code is \$0D, so if you create the file on Windows, remove the \$0A from \$0D and \$0A before transferring it to Flex.

(2) How to write function arguments:

Since it is an older compiler, it uses the KR style where the argument type is written on the next line.

Example: main (argc, argv)

```
    int argc;  
    char *argv[];  
    {  
  
    }
```

There are many aspects that make it difficult to use, such as having to write in the old style and the unhelpful error messages, but I think it is convenient when you want to write utilities with file I/O in a short amount of time.

That's all, but I can't really confirm which Linux distributions and versions can create these files, so I've uploaded both the 31KB MC09.CMD and 29KB MC09O.CMD files I created, along with my own command LISTN.C and MOT2CMD.C as examples of older style C files, to [Microsoft OneDrive](#).

